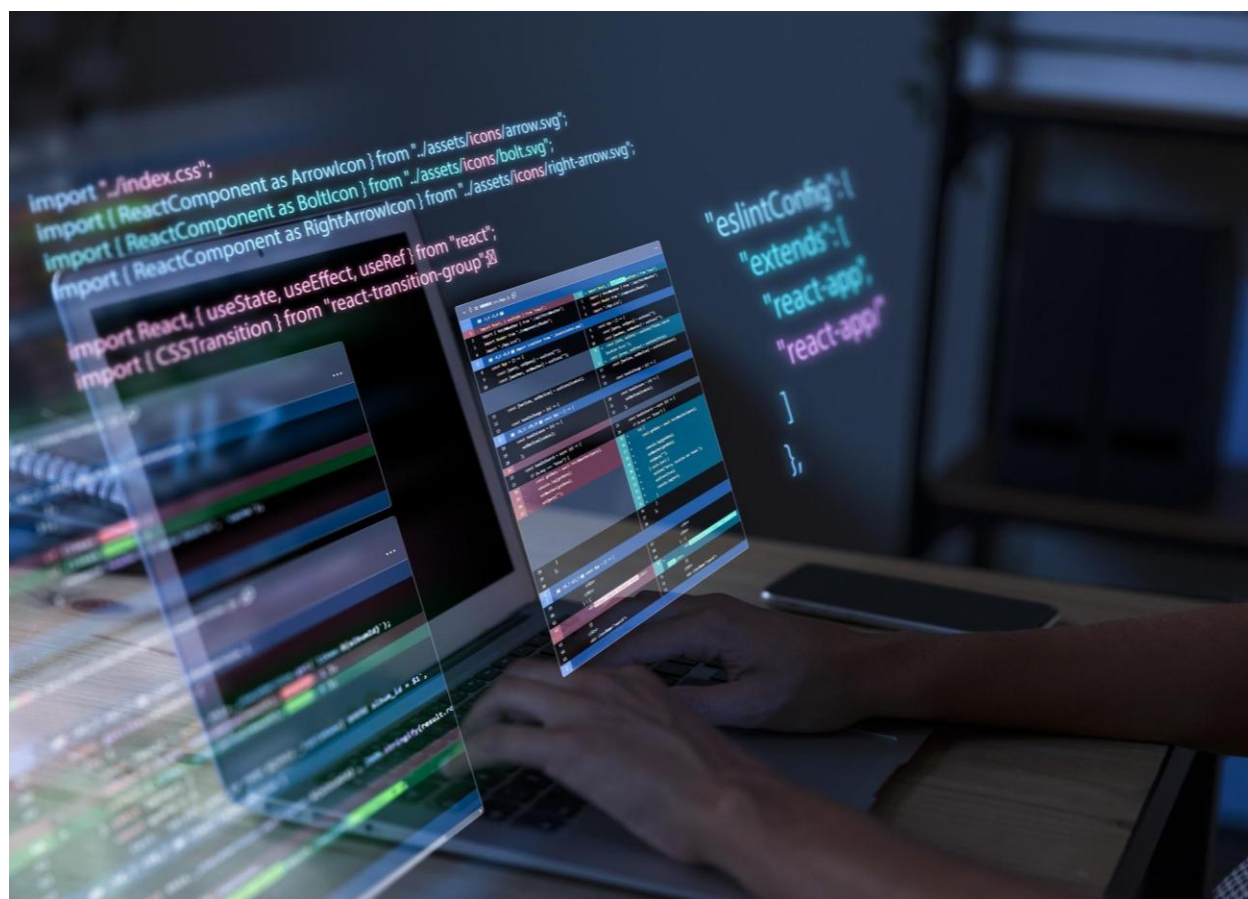


Ejercicios

Colecciones (IV)



Autor: Marcela Martín

Fecha: abril 2026

1. Crea un sistema de Gestión de Estudiantes con Orden Natural

a) Clase **Estudiante**

- i. Atributos
 - nombre (String)
 - apellido (String)
 - nota (double).
- ii. Implementa la interfaz Comparable en la clase Estudiante para que los estudiantes se ordenen alfabéticamente por su apellido y, en caso de apellidos iguales, por su nombre.

b) Clase **GestionEstudiantes**.

- método agregarEstudiante que reciba un ArrayList de Estudiante y un
 - nuevo objeto Estudiante. Este método deberá agregar el nuevo estudiante a la lista.
- método mostrarEstudiantes que reciba un ArrayList de Estudiante e imprima por consola la información de cada estudiante (nombre, apellido y nota).
- Método main:
 - crea un ArrayList de Estudiante y agrega al menos 5 estudiantes diferentes.
 - Luego, utiliza el método Collections.sort() para ordenar la lista de estudiantes y
 - finalmente, utiliza el método mostrarEstudiantes para verificar el orden.

2. Evolución del ejercicio 1. Orden por Nota

- a) Crea una nueva clase llamada **ComparadorEstudiantesPorNota** que implemente la interfaz Comparator<Estudiante>. Esta clase compara dos objetos Estudiante basándose en su nota de forma descendente (de mayor a menor nota).

b) En la clase **GestionEstudiantes**,

- i. Crea un método llamado **ordenarPorNota** que reciba un ArrayList de Estudiante y utilice la instancia de ComparadorEstudiantesPorNota para ordenar la lista.

- ii. Modifica el método **agregarEstudiante** para que lance una excepción personalizada y verificada llamada **NotaInvalidaException** si la nota del estudiante que se intenta agregar es menor que 0 o mayor que 10.
- c) En la APP:
 - i. dentro de un bloque try-catch, intenta agregar un estudiante con una nota inválida.
 - ii. Si se lanza la excepción **NotaInvalidaException**, captura la excepción e imprime un mensaje de error informativo.
 - iii. Después de intentar agregar el estudiante inválido, ordena la lista de estudiantes por nota utilizando el método **ordenarPorNota** y muestra la lista ordenada.
- 3. Se quiere mantener un ranking de videojuegos basado en su puntuación y año de lanzamiento.
 - a) **Videojuego** con atributos: titulo, puntuacion (0 a 100), y anyo.
 - b) Implementa **Comparable<Videojuego>** para que se ordenen por **puntuación descendente**.
 - c) Crea un **Comparator** para ordenar por **año ascendente** y, en caso de empate, por Título.
 - d) En la App crea un **ArrayList** de Videojuegos y prueba el orden por puntuación y el orden por año – título
- 4. Se registran distintos eventos meteorológicos ocurridos durante el año.
 - a) **EventoClimatico** con atributos: tipo (tormenta, nevada, ola de calor...), fecha (**LocalDate**), intensidad (1 a 10).
 - b) Implementa **Comparable<EventoClimatico>** para que se ordenen por **fecha**.
 - c) Crea un **Comparator** para ordenar por **intensidad descendente** y luego por tipo.
 - d) En la App crea una lista de eventos climáticos y prueba el orden por fecha y el orden por intensidad-tipo.
- 5. Se quiere mantener una base de datos de juegos de mesa disponibles.
 - a) Clase **JuegoMesa** con atributos: nombre, duracionMinutos, numJugadores (mínimo número recomendado).
 - b) Implementa **Comparable<JuegoMesa>** para ordenar por duración.

- c) Crea un Comparator que ordene por número de jugadores y, si empatan, por nombre.
 - d) En la App crea una lista de juegos y prueba el orden por duración y el orden por cantidad de jugadores-nombre
6. Se quiere registrar una lista de reproducción musical que se puede ordenar de distintas formas.
- a) Clase **Cancion** con: titulo, artista, duracionSegundos.
 - b) Implementa Comparable<Cancion> para ordenar por título alfabéticamente.
 - c) Crea un Comparator para ordenar por duración descendente y luego por artista.
 - d) En la App crea una lista de canciones y prueba elorden por título y el orden por duración-artista